Overview of Statistical Learning and Modeling – 36-600

Lab 11T: Nonlinear Regression
Due Wednesday, November 12, 11:59pm

Trishla Nair

```
suppressMessages(library(tidyverse))
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

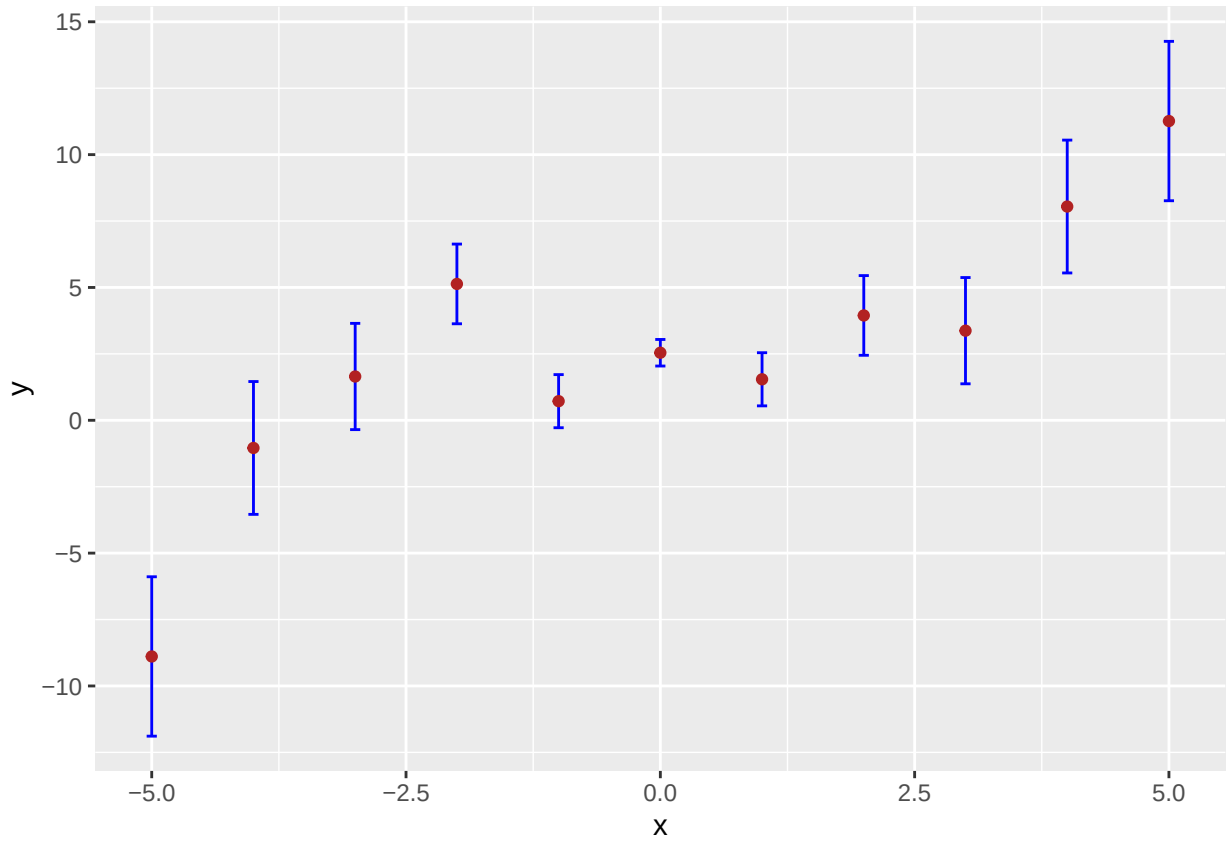
Data

Let's start by simulating a dataset from a nonlinear curve:

```
set.seed(555)
x <- -5:5
y <- 0.1*x^3 - 0.5*x + 2.1 + rnorm(length(x), mean = 0, sd = 0.5*(1 + abs(x)))
e <- 0.5*(1 + abs(x))

df <- data.frame("x" = x, "y" = y, "e" = e)

suppressMessages(library(tidyverse))
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick")
```



With this data, you'll be able to see exactly how different methods perform differently on the same comprehensible dataset.

Questions

Question 1

Implement a (weighted) global cubic polynomial regression model in a similar manner to that implemented in the notes; more specifically, this means: learn the model, run `predict()` to determine the regression line, plot the data with the regression line superimposed, show the coefficients, and compute the mean-squared error. Just like the notes, don't worry about splitting the data into training and test sets.

```
pr.out  <- lm(y ~ poly(x, 3, raw = TRUE), data = df,
              weights = 1/(e^2))
pr.pred <- predict(pr.out)
mean((y - pr.pred)^2)
```

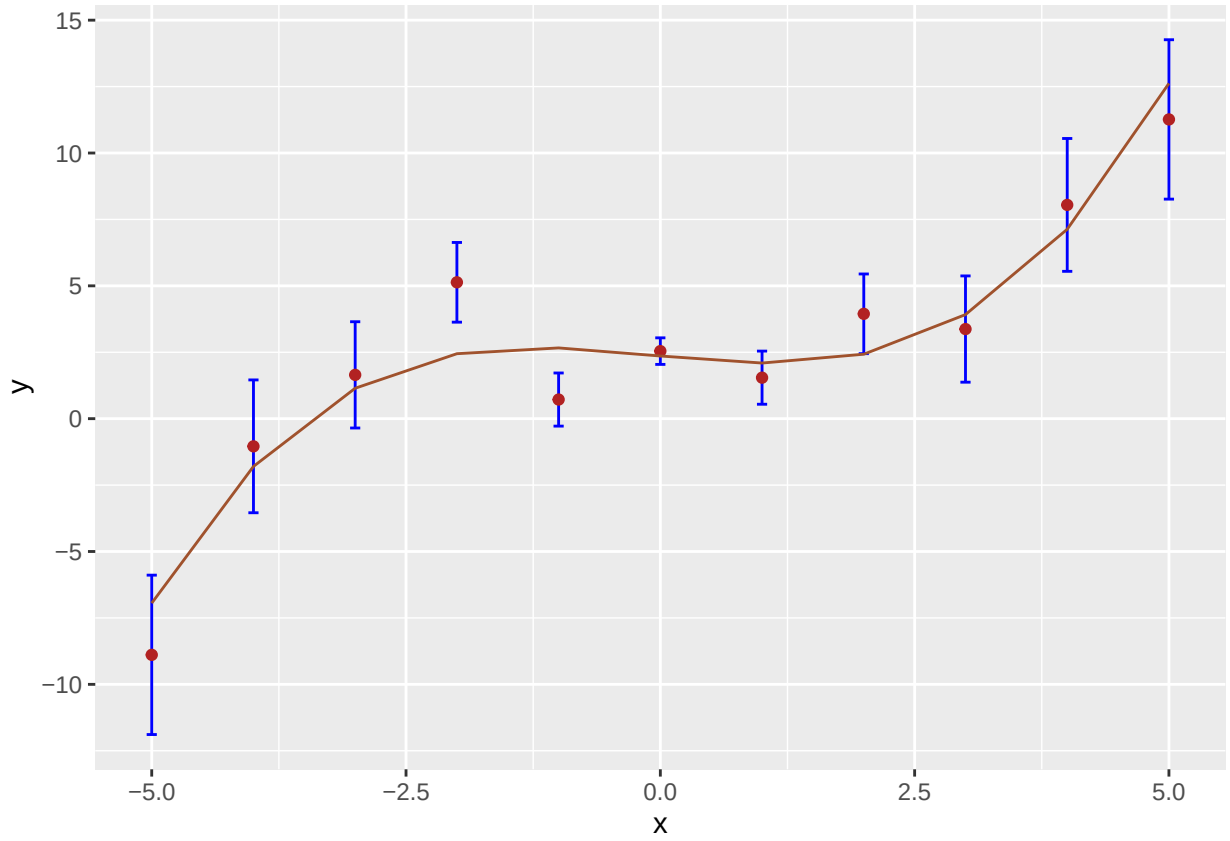
```
## [1] 1.930721
```

```
data.frame("Estimate" =
             summary(pr.out)$coefficients[, 1])
```

```
##                             Estimate
```

```
## (Intercept)              2.35941485
## poly(x, 3, raw = TRUE)1 -0.37839403
## poly(x, 3, raw = TRUE)2  0.01917133
## poly(x, 3, raw = TRUE)3  0.09339114
```

```
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick") +
  geom_line(data = data.frame(x, "y" = pr.pred),
            mapping = aes(x = x, y = y), col = "sienna")
```



Question 2

Repeat Question 1, but using a regression splines model. Assume four degrees of freedom.

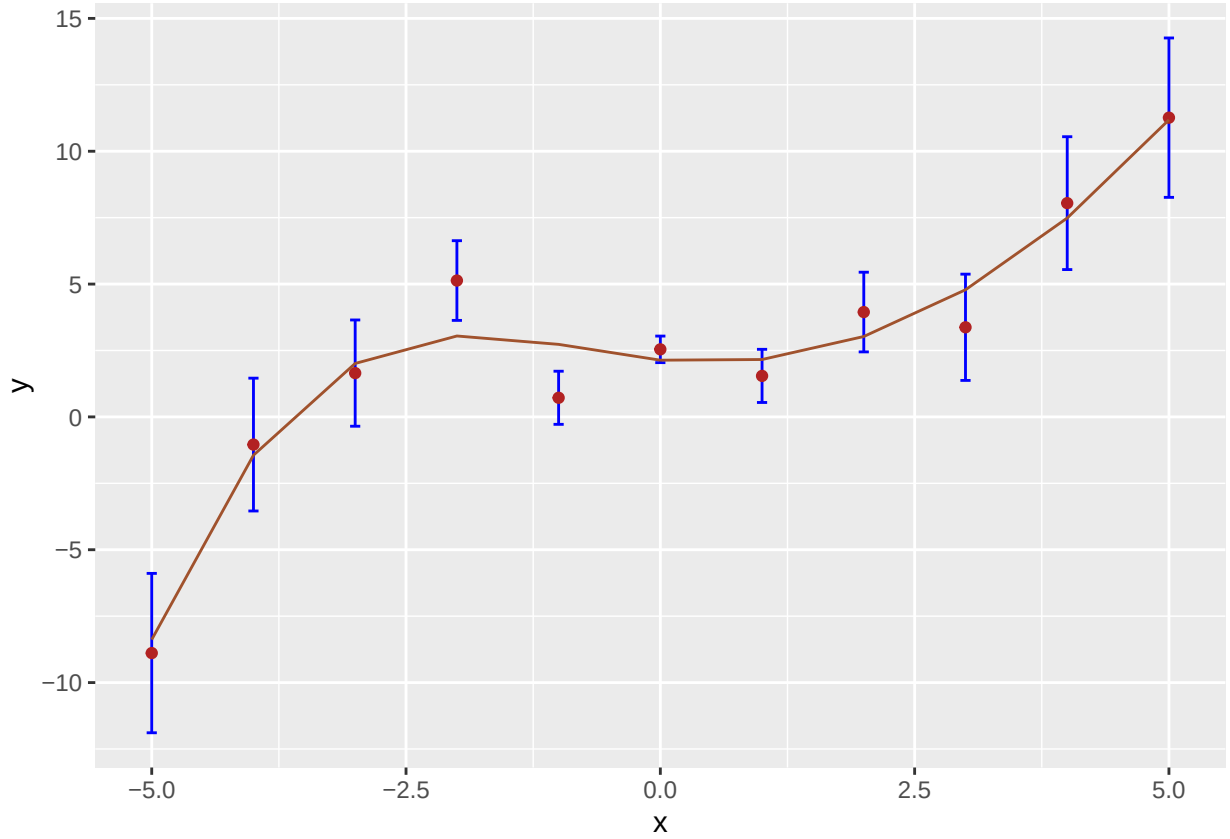
```
library(splines)
# Default: cubic spline -> K+2 basis functions
#  remember: K is the # of non-overlapping segments
s.out  <- lm(y ~ bs(x, df = 4), data = df,weights = 1/(e^2))
s.pred <- predict(s.out)
mean((y - s.pred)^2)
```

```
## [1] 1.150384
```

```
data.frame("Estimate" =
             summary(s.out)$coefficients[, 1])
```

```
##                  Estimate
## (Intercept)    -8.381681
## bs(x, df = 4)1 15.069067
## bs(x, df = 4)2  7.226883
## bs(x, df = 4)3 12.546440
## bs(x, df = 4)4 19.563844
```

```
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick") +
  geom_line(data = data.frame(x, "y" = s.pred),
            mapping = aes(x = x, y = y), col = "sienna")
```



Question 3

Repeat Question 1, but with a smoothing spline model. Note that you may get a "surprising" result.

```
ss.out  <- suppressWarnings(
  smooth.spline(df$x, y = df$y,
                w = 1/(df$e^2), cv = TRUE)
```

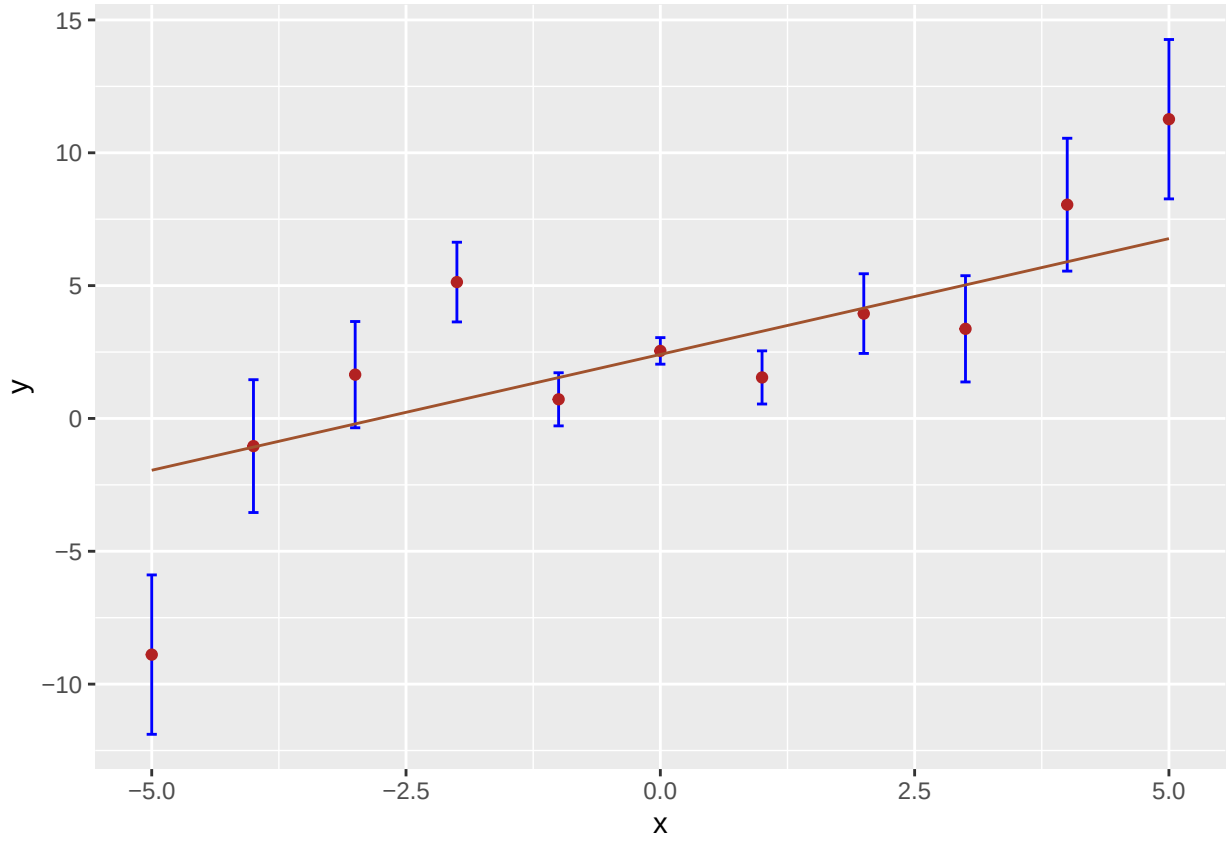
```
)
ss.out$lambda
```

```
## [1] 21221.01
```

```
ss.pred <- predict(ss.out)
mean((y - ss.pred$y)^2)
```

```
## [1] 9.351152
```

```
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick") +
  geom_line(data = data.frame(x, "y" = ss.pred$y),
            mapping = aes(x = x, y = y), col = "sienna")
```



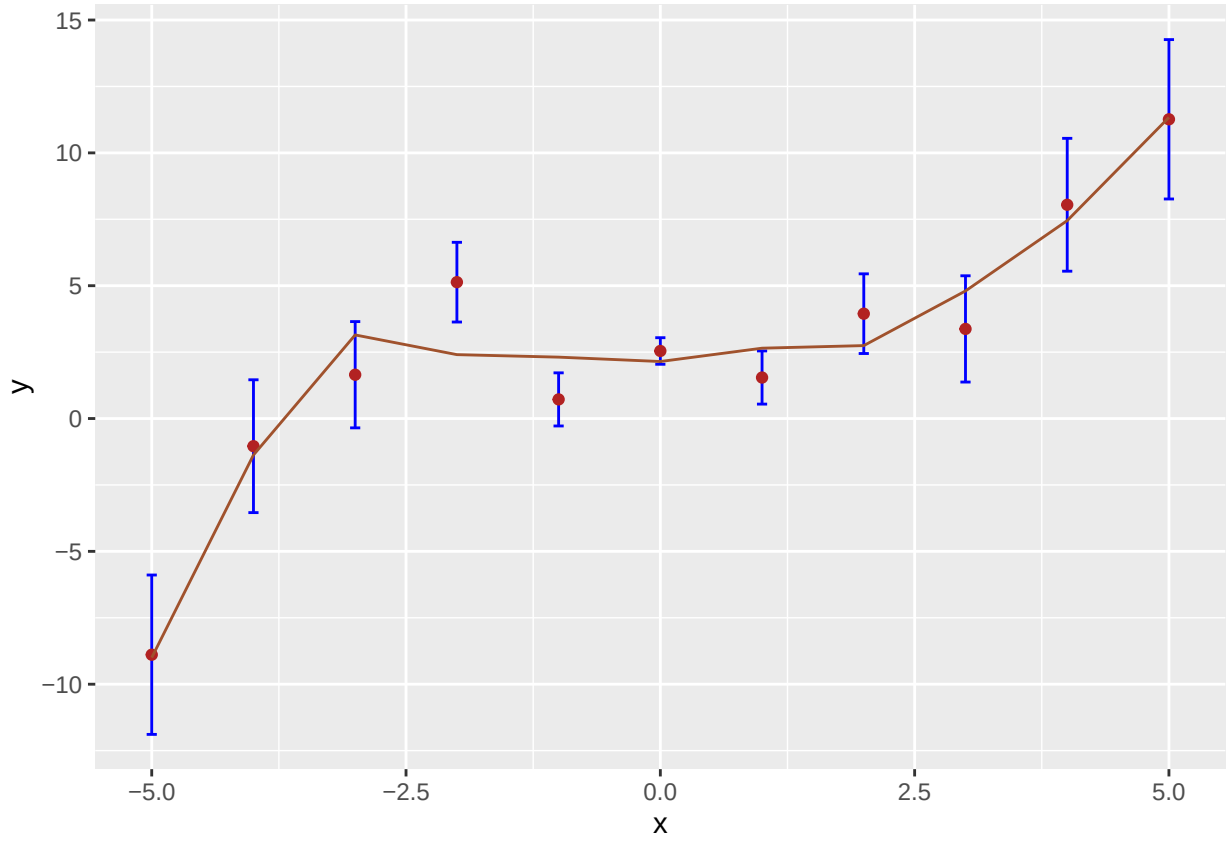
Question 4

Repeat Question 1, but with a local polynomial regression model. Assume a `span` of 0.6.

```
lpr.out  <- loess(y ~ x, data = df,
                  weights = 1/(df$e)^2, span = 0.6)
lpr.pred <- predict(lpr.out)
mean((y - lpr.pred)^2)
```

```
## [1] 1.596355
```

```
ggplot(data = df, mapping = aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y-e, ymax = y+e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick") +
  geom_line(data = data.frame(x, "y" = lpr.pred),
            mapping = aes(x = x, y = y), col = "sienna")
```



Question 5

Reform the plot from Question 4, but add a one-standard-error confidence band. You can do this by running the first line, then subsequently adding the last two lines onto your `ggplot()` call, once they're specified as you prefer:

```
p <- predict(lpr.out, se = TRUE)

+ geom_line(mapping = aes(x = lpr.out$x, y = lpr.out$fitted + p$se),
            color = "[your color]", linetype = "dashed")
+ geom_line(mapping = aes(x = lpr.out$x, y = lpr.out$fitted - p$se),
            color = "[your color]", linetype = "dashed")
```

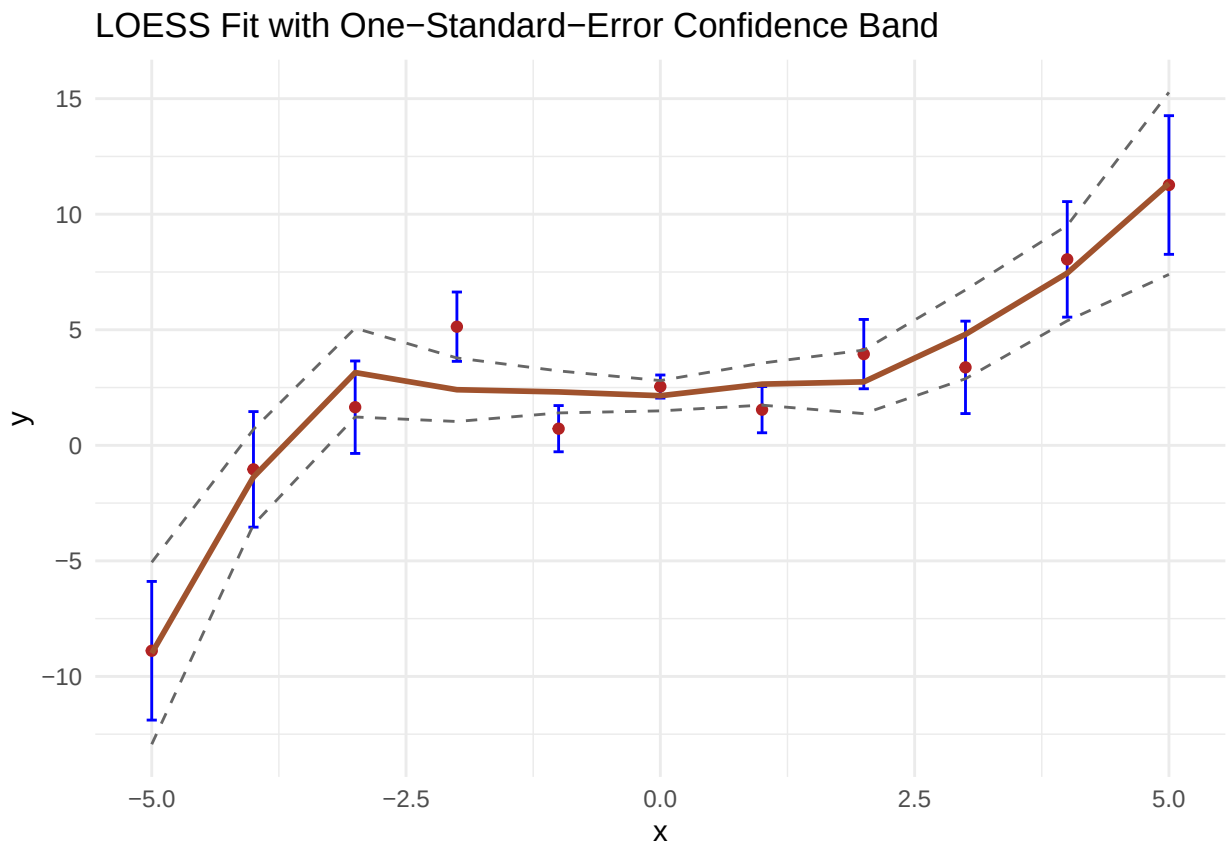
What does the band actually mean? Because it's a one-standard-error band, it means that for any given x, there is an approximately 68% chance that the band overlaps the true underlying function value. This is generally a rough statement, though, given that *there is almost always correlation between neighboring data*

points (i.e., the lack of independence between y_{i-1}, y_i, and y_{i+1}, etc.). Just think of the band as a notion of how uncertain your fitted curve is at each x: is the band wide or narrow? Note that the bands get wider as we get to either end of the data: this is an expected feature, not a bug. There's fewer data within the span at either end, so the fitted function is that much more uncertain towards the periphery.

```
lpr.out  <- loess(y ~ x, data = df, weights = 1/(df$e)^2, span = 0.6)
p <- predict(lpr.out, se = TRUE)
pred.df <- data.frame(
  x = lpr.out$x,
  fit = lpr.out$fitted,
  upper = lpr.out$fitted + p$se,
  lower = lpr.out$fitted - p$se
)

ggplot(data = df, aes(x = x, y = y)) +
  geom_errorbar(aes(ymin = y - e, ymax = y + e), width = 0.1, color = "blue") +
  geom_point(color = "firebrick") +
  geom_line(data = pred.df, aes(x = x, y = fit), color = "sienna", size = 1) +
  geom_line(data = pred.df, aes(x = x, y = upper), color = "gray40", linetype = "dashed") +
  geom_line(data = pred.df, aes(x = x, y = lower), color = "gray40", linetype = "dashed") +
  theme_minimal() +
  labs(
    title = "LOESS Fit with One-Standard-Error Confidence Band",
    y = "y",
    x = "x"
  )
```

```
## Warning: Using ‘size‘ aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use ‘linewidth‘ instead.
## This warning is displayed once every 8 hours.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```


LOESS Fit with One−Standard−Error Confidence Band
15
10
5
0
−5
−10
y
−5.0
−2.5
0.0
2.5
5.0
x